LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASICS WITH COMPOSE

Oleh:

Ashma Salimah NIM. 2410817220015

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Ashma Salimah
NIM : 2410817220015

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 0
DAFTAR ISI 1
DAFTAR GAMBAR 2
DAFTAR TABEL 3
SOAL PRAKTIKUM 5
A. Source Code 8
B. Output Program 14
C. Pembahasan 17
D. Essay 20
E. Tautan Git 25

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 5
Gambar 2. Tampilan Dadu Setelah Di-Roll 6
Gambar 3. Tampilan Roll Dadu Double 6
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal 14
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung 14
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double 15
Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal 15
Gambar 8. Screenshot Hasil Jawaban Soal Compose Belum Beruntung 16
Gambar 9. Screenshot Hasil Jawaban Soal Compose Dadu Double 16

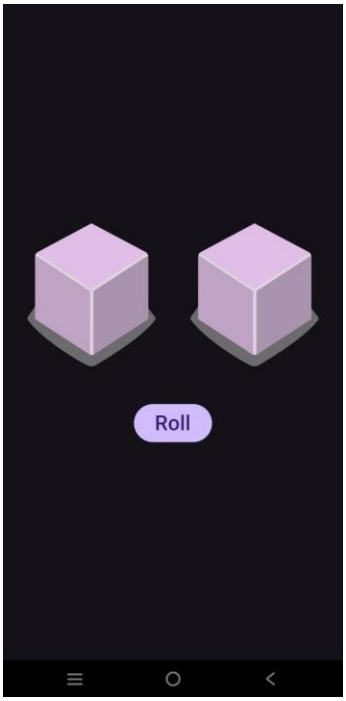
DAFTAR TABEL

Tabel 1. Source Code Main Activity.kt XML 8
Tabel 2. Source Code Main Activity.kt Compose 9
Tabel 3. Source Code activity_main.xml 12

SOAL 1

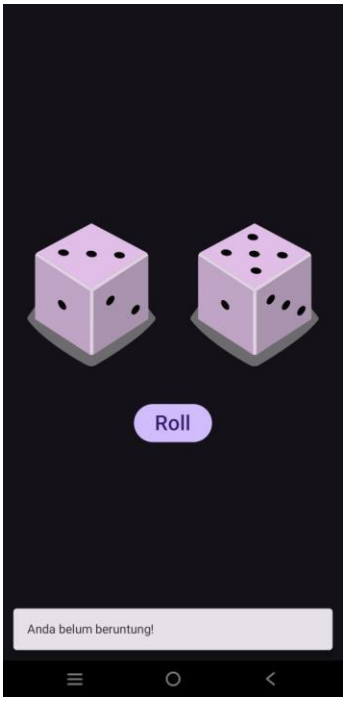
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



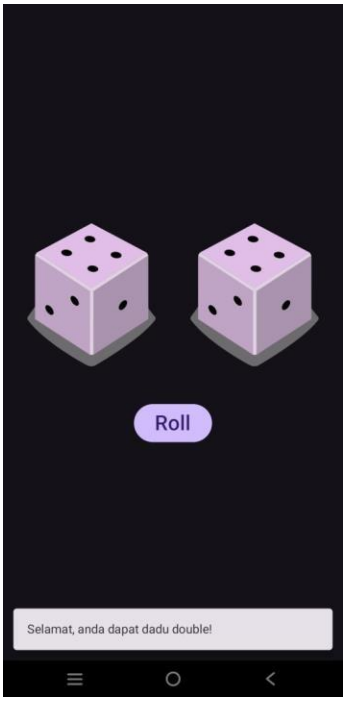
Gambar 1. Tampilan Awal Aplikasi

2. Tampilan Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan "Selamat, anda dapat dadu double!" seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

A. Source Code

Tabel 1. Source Code Main Activity.kt XML

No	Main Activity.kt (XML)
1	package com.example.prakmodul1
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import android.widget.TextView
7	import androidx.appcompat.app.AppCompatActivity
8	
9	class XmlActivity : AppCompatActivity() {
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	setContentView(R.layout.*activity_xml*)
13	
14	val dice1 = findViewById<ImageView>(R.id.*imageViewDice1*)
15	val dice2 = findViewById<ImageView>(R.id.*imageViewDice2*)
16	val button = findViewById<Button>(R.id.*button*)
17	val message = findViewById<TextView>(R.id.*textViewMessage*)
18	
19	button.setOnClickListener **{**
20	val result1 = (1..6).*random*()
21	val result2 = (1..6).*random*()
22	
23	dice1.setImageResource(when (result1) {
24	1 -> R.drawable.*dice_1*
25	2 -> R.drawable.*dice_2*
26	3 -> R.drawable.*dice_3*
27	4 -> R.drawable.*dice_4*
28	5 -> R.drawable.*dice_5*
29	else -> R.drawable.*dice_6*
30	})
31	
32	dice2.setImageResource(when (result2) {
33	1 -> R.drawable.*dice_1*
34	2 -> R.drawable.*dice_2*
35	3 -> R.drawable.*dice_3*
36	4 -> R.drawable.*dice_4*
37	5 -> R.drawable.*dice_5*
38	else -> R.drawable.*dice_6*
39	})
40	

```
            message.text = if (result1 == result2) "Selamat, anda dapat dadu double!"
            else "Anda belum beruntung!"

            message.visibility = TextView.VISIBLE
        }
    }
}
```

Tabel 2. Source Code Main Activity.kt Compose

No	Main Activity.kt (Compose)
1	package com.example.prakmodul1
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.Spacer
10	import androidx.compose.foundation.layout.fillMaxSize
11	import androidx.compose.foundation.layout.height
12	import androidx.compose.foundation.layout.wrapContentSize
13	import androidx.compose.foundation.layout.Arrangement
14	import androidx.compose.material3.Button
15	import androidx.compose.material3.Text
16	import androidx.compose.runtime.Composable
17	import androidx.compose.runtime.getValue
18	import androidx.compose.runtime.mutableStateOf
19	import androidx.compose.runtime.remember
20	import androidx.compose.runtime.setValue
21	import androidx.compose.ui.Alignment
22	import androidx.compose.ui.Modifier
23	import androidx.compose.ui.res.painterResource
24	import androidx.compose.ui.unit.dp
25	import androidx.compose.foundation.background
26	import androidx.compose.foundation.layout.Box
27	import androidx.compose.foundation.layout.fillMaxWidth
28	import androidx.compose.foundation.layout.padding
29	import androidx.compose.ui.unit.sp
30	import androidx.compose.material3.ButtonDefaults
31	import androidx.compose.ui.graphics.Color
32	import com.example.prakmodul1.R
33	import com.example.prakmodul1.ui.theme.DiceRollerTheme

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
{
        super.onCreate(savedInstanceState)
        setContent {
            DiceRollerTheme {
                DiceRollerApp()
            }
        }
    }
}

@Composable
fun DiceRollerApp() {
    DiceRollerScreen()
}

@Composable
fun DiceRollerScreen(modifier: Modifier = Modifier.
fillMaxSize() .wrapContentSize(Alignment.Center)) {
    var result1 by remember {mutableStateOf(0)}
    var result2 by remember {mutableStateOf(0)}
    var message by remember { mutableStateOf("") }

    val imageResource1 = when (result1) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    val imageResource2 = when (result2) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Box(modifier = modifier) {
        Column(
            modifier = Modifier.fillMaxSize(),
```

```
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Row {
                Image(
                    painter = painterResource(imageResource1),
                    contentDescription = result1.toString()
                )
                Spacer(modifier = Modifier.height(8.dp))
                Image(
                    painter = painterResource(imageResource2),
                    contentDescription = result2.toString()
                )
            }
            Spacer(modifier = Modifier.height(16.dp))
            Button(
                onClick = {
                    result1 = (1..6).random()
                    result2 = (1..6).random()
                    message = if (result1 == result2)
                        "Selamat, anda dapat dadu double!"
                    else
                        "Anda belum beruntung!"
                },
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFFD0BCFF)
                )
            ) {
                Text("Roll", color = Color.Black)
            }
        }

        if (message.isNotEmpty()) {
            Text(
                text = message,
                modifier = Modifier
                    .align(Alignment.BottomCenter)
                    .fillMaxWidth()
                    .padding(16.dp)
                    .background(Color.White)
```

```
                    .padding(13.dp),
                color = Color.Black,
                fontSize = 15.sp
            )
        }
    }
}
```

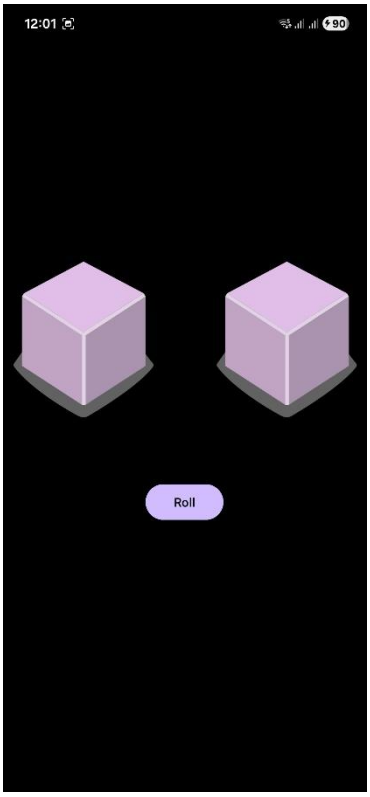
Tabel 3. Source Code activity_main.xml

No	activity_main.xml

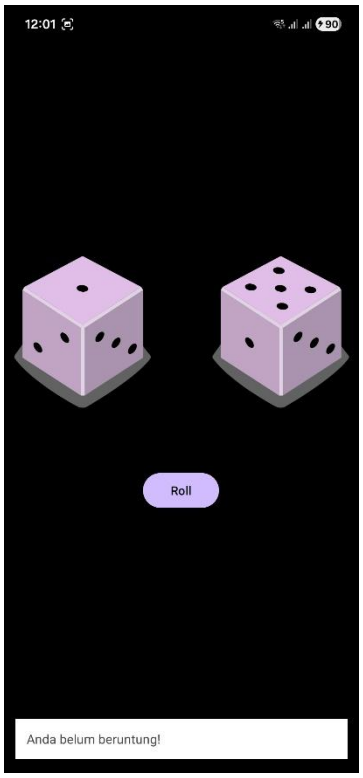
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout

xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".XmlActivity">

    <ImageView
        android:id="@+id/imageViewDice1"
        android:layout_width="230dp"
        android:layout_height="230dp"
        android:src="@drawable/dice_0"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toStartOf="@+id/imageViewDice2"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintVertical_bias="0.4"/>

    <ImageView
        android:id="@+id/imageViewDice2"
        android:layout_width="230dp"
        android:layout_height="230dp"
        android:src="@drawable/dice_0"
app:layout_constraintStart_toEndOf="@+id/imageViewDice1"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintVertical_bias="0.4"/>

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:backgroundTint="#D0BCFF"
        android:text="Roll"
        android:textColor="#000000"
        app:cornerRadius="50dp"
app:layout_constraintBottom_toTopOf="@+id/textViewMessage"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/imageViewDice2"
        app:layout_constraintVertical_bias="0.167" />

    <TextView
        android:id="@+id/textViewMessage"
        android:layout_width="385dp"
        android:layout_height="47dp"
        android:layout_marginStart="16dp"
        android:layout_marginEnd="16dp"
        android:layout_marginBottom="16dp"
        android:background="#FFFFFF"
        android:padding="13dp"
        android:text=""
        android:textSize="15sp"
        android:visibility="invisible"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

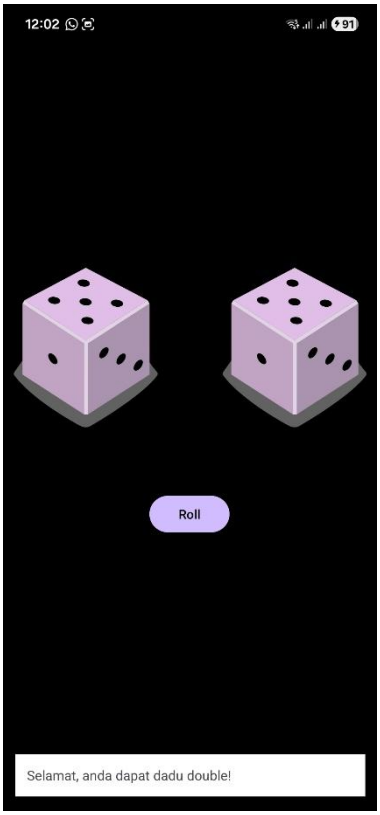
B. Output Program



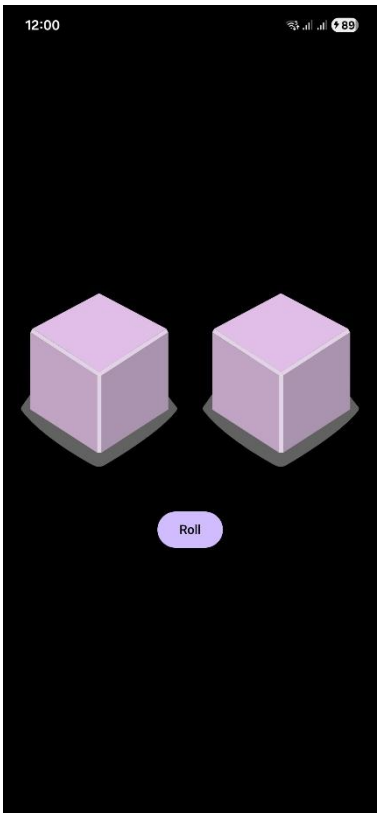
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal



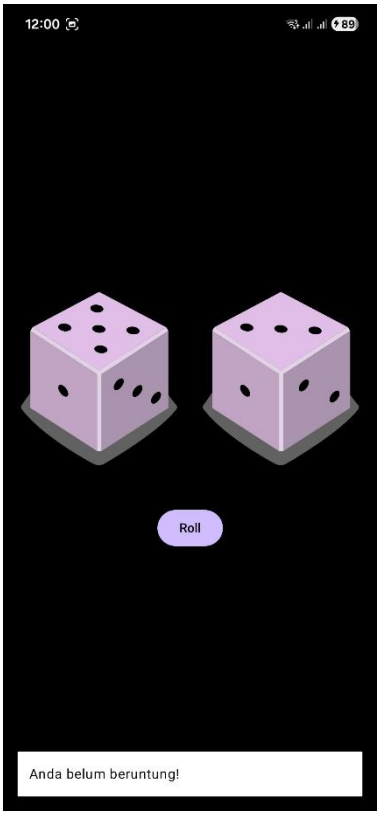
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung



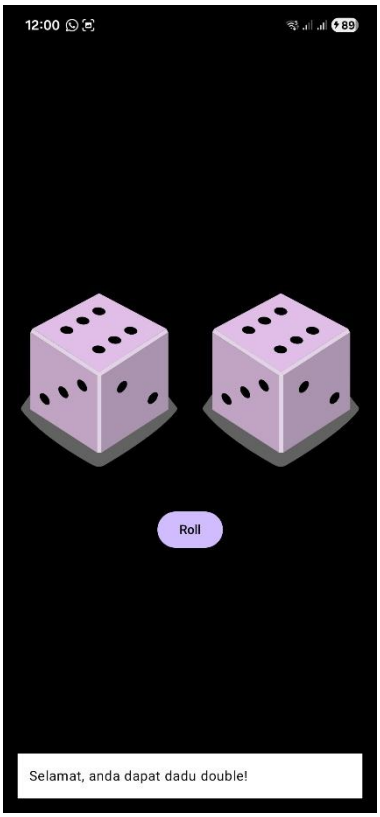
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double



Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal



Gambar 8. Screenshot Hasil Jawaban Soal Compose Belum Beruntung



Gambar 9. Screenshot Hasil Jawaban Soal Compose Dadu Double

C. Pembahasan

XmlActivity.kt XML

File XmlActivity.kt adalah pengendali utama aplikasi DiceRoller versi XML. Di sinilah semua logika ditulis, mulai dari menghubungkan tampilan ke kode, mengambil komponen-komponen dari file XML, sampai mengatur apa yang terjadi saat tombol Roll ditekan. Saat aplikasi dibuka, dua dadu kosong ditampilkan, dan setelah tombol Roll ditekan, dadu akan menampilkan angka secara random. Jika kedua dadu menghasilkan angka yang sama maka muncul pesan pada bagian bawah bahwa pengguna dapat dadu double dan begitu juga jika angka kedua dadu berbeda akan muncul pesan pada bagian bawah layar.

`import` berfungsi untuk mengimpor library yang dibutuhkan seperti `AppCompatActivity` untuk fitur dasar Android, `Button`, `ImageView`, dan `TextView` untuk mengambil dan mengontrol komponen tampilan.

`class XmlActivity : AppCompatActivity()` berfungsi untuk membuat kelas utama bernama `Activity` yang mewarisi `AppCompatActivity`, sehingga aplikasi bisa menampilkan layar dan mengelola kondisi-kondisi yang dialami aplikasi dari pertama dibuka sampai ditutup.

`override fun onCreate` merupakan fungsi yang otomatis dijalankan pertama kali saat aplikasi dibuka, digunakan untuk setup awal seperti menghubungkan tampilan dan mengatur listener.

`setContentView(R.layout.activity_xml)` berfungsi untuk menghubungkan file tampilan `activity_xml.xml` dengan file Kotlin ini, sehingga komponen yang ada di XML bisa digunakan di sini.

`findViewById` berfungsi untuk mengambil komponen dari file XML berdasarkan id-nya, yaitu dua gambar dadu, tombol Roll, dan teks pesan agar bisa dikendalikan dari kode.

`button.setOnClickListener` berfungsi untuk mendeteksi saat tombol Roll ditekan dan menjalankan kode di dalamnya.

`(1..6).random()` berfungsi untuk menghasilkan angka acak antara 1 sampai 6 untuk masing-masing dadu.

`setImageResource(when {...})` berfungsi untuk mengubah gambar dadu sesuai angka yang didapat. `when` digunakan untuk menentukan gambar mana yang ditampilkan berdasarkan nilai angkanya.

message.text = if (result1 == result2) berfungsi untuk menentukan pesan yang ditampilkan, jika kedua dadu sama maka akan keluar pesan "Selamat, anda dapat dadu double!" dibawah layar, jika dua angka dadu berbeda maka "Anda belum beruntung!".

message.visibility = TextView.VISIBLE berfungsi untuk memunculkan teks pesan yang sebelumnya tersembunyi, baru ditampilkan setelah tombol Roll ditekan.

activity_main.xml

File ini mengatur tampilan visual aplikasi android versi XML. Di sini didefinisikan letak dan tampilan semua komponen yang terlihat di layar, yaitu dua gambar dadu di tengah, tombol Roll di bawah dadu, dan teks pesan di paling bawah layar. File ini tidak berisi logika apapun, hanya mengatur tampilan yang nanti logikanya akan dikendalikan oleh `XmlActivity.kt`.

`ConstraintLayout` merupakan wadah utama yang menampung semua komponen tampilan yang memenuhi seluruh layar.

`ImageView`, yaitu `imageViewDice1` dan `imageViewDice2` yaitu dua gambar dadu berukuran `230dp` yang menampilkan dadu kosong saat pertama dibuka. Diposisikan berdampingan di tengah layar dengan posisi sedikit ke atas menggunakan `constraintVertical_bias="0.4"`.

`Button` merupakan tombol dengan tulissan "Roll" berwarna ungu muda dengan sudut membulat, diposisikan di antara gambar dadu dan teks pesan. Saat ditekan akan menjalankan logika di `XmlActivity.kt`.

`TextView` merupakan teks pesan yang diposisikan di paling bawah layar dengan jarak dari tepi. `visibility="invisible"` berfungsi untuk membuat teks tidak terlihat saat aplikasi pertama dibuka, dan baru muncul setelah tombol Roll ditekan.

MainActivity.kt Compose

File ini adalah implementasi aplikasi dadu menggunakan Jetpack Compose. Berbeda dengan versi XML yang memisahkan tampilan UI dan logika pada dua file berbeda, di sini semuanya ditulis dalam satu file Kotlin. File ini akan menampilkan dua dadu kosong saat pertama dibuka, lalu mengacak nilai dadu dan menampilkan pesan saat tombol Roll ditekan. Bedanya, di Compose tampilan otomatis ikut berubah saat data berubah tanpa perlu diatur satu per satu secara manual.

`override fun onCreate` merupakan fungsi yang dijalankan pertama kali saat aplikasi dibuka. Kemudian `setContent` digunakan agar langsung menampilkan UI menggunakan Compose.

`DiceRollerTheme` berfungsi untuk membungkus seluruh tampilan dengan tema aplikasi agar warna dan gaya tampilan konsisten.

`DiceRollerApp()` berfungsi untuk memanggil `DiceRollerScreen()` sebagai tampilan utama aplikasi.

`remember { mutableStateOf() }` berfungsi untuk menyimpan nilai dadu pertama, dadu kedua, dan pesan hasil.

`when (result1/result2)` berfungsi untuk menentukan gambar dadu yang ditampilkan berdasarkan nilai angkanya, dari dadu nol atau `dice_0` saat awal hingga dadu dengan angka 1 sampai 6.

`Box` adalah wrapper utama yang befungsi untuk membuat penempatan elemen secara bertumpuk, box ini digunakan untuk memposisikan teks pesan bisa berada di paling bawah layar.

`Column` berfungsi untuk menyusun elemen secara vertikal dari atas ke bawah dengan posisi di tengah layar.

`Row` berfungsi untuk menampilkan dua gambar dadu berdampingan secara horizontal.

`Button` merupaka tombol bertulisan “Roll” berwarna ungu muda. Saat ditekan, kedua nilai dadu berubah menjadi angka random dan pesan akan diperbarui secara otomatis sesuai penjikaan karena Compose bersifat reaktif.

`Text` berfungsi untuk menampilkan pesan hasil roll di bagian bawah layar dalam kotak putih. `message.`*`visibility`*` = TextView.`*`VISIBLE`* berfungsi untuk membuat teks tidak terlihat saat aplikasi pertama dibuka, dan baru muncul setelah tombol Roll ditekan.

D. Essay

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

Jawab :

Imperative programming adalah paradigma pemrograman yang berfokus pada bagaimana sebuah program diselesaikan, menggunakan serangkaian instruksi atau perintah eksplisit langkah demi langkah. Sedangkan declarative programming adalah paradigma pemrograman yang berfokus pada apa yang ingin dicapai oleh hasil akhir, menyembunyikan detail alur kontrol.

Further, if the developer was looking for parts of the UI to always change while the app was being used (for example, showing/hiding buttons programmatically, or changing layouts based on user action), it required lots and lots of extra code to change the app at runtime following the original XML declaration [1].

Kalimat di atas ini menjelaskan bahwa untuk pembuatan UI di Android tradisional yaitu xml menggunakan paradigma imperatif. Pengembang harus menulis banyak kode ekstra secara manual untuk memperbarui atau mengubah tampilan saat aplikasi berjalan. Proses pembaruan UI ini dilakukan dengan memberikan instruksi teknis langkah demi langkah secara spesifik. Misalnya jika kita ingin menyembunyikan tombol secara terprogram.

Declarative UI Model: Simplifies UI logic by focusing on what the UI should look like, not how to update it [1].

Sedangkan dari kalimat di atas secara langsung mendefinisikan UI Android menggunakan deklaratif programming pada Jetpack Compose. Paradigma ini menyederhanakan logika dengan cara membiarkan pengembang berfokus hanya pada apa bentuk antarmuka yang diinginkan, tanpa perlu memikirkan instruksi detail mengenai bagaimana cara UI tersebut harus diperbarui.

Declarative UI requires thinking about app development from a new perspective, accepting that it's okay to rebuild parts of your UI from scratch instead of modifying them [2].

Kemudian pada artikel selanjutnya dari kalimat "*instead of modifying them*" artikel ini menyebutkan bagaimana pemrograman imperatif lama bekerja. Pada pemrograman imperatif yang lama, pengembang terbiasa memodifikasi atau mengubah bagian-bagian antarmuka secara manual ketika ada perubahan data, yang mana cara ini dianggap lebih rentan terhadap masalah ketidaksinkronan.

In a declarative framework, changes in state trigger a rebuild of the UI, which removes a whole category of state-related bugs and makes it easier to reason about updates. You describe what the UI should look like for a given state, and the framework figures out how to do it using sensible defaults and context [2].

Kalimat di atas ini memberitahu bahwa pada kerangka pemrograman deklaratif, perubahan tampilan diubah secara otomatis oleh perubahan status atau *state*. Daripada mengubah tampilan secara manual, framework akan membangun ulang UI tersebut berdasarkan status terbaru, sehingga menghilangkan banyak bug.

Menurut saya memilih menggunakan deklaratif programming ataupun imperative programming itu tergantung dari apa program yang kita buat. Tetapi tidak menutup kemungkinan bahwa saya juga menyetujui opini bahwa deklaratif programming memberikan struktur yang jauh lebih ringkas untuk kenyamanan developer, dengan memperbarui UI secara otomatis sesuai status.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

Kotlin can have a primary constructor as well as a secondary constructor... This secondary constructor feature is absent in Java. The utility of secondary constructor is that it reduces the lines of code [3].

Kalimat di atas ini menjelaskan bahwa bahasa pemrograman Kotlin menyederhanakan inisialisasi kelas atau OOP dengan menyediakan primary dan secondary constructor. Fitur ini tidak ada di Java, sehingga pengembang Java harus menggunakan metode constructor

overloading yang panjang. Penggunaan Kotlin membuat baris kode untuk mendefinisikan sebuah kelas menjadi jauh lebih sedikit dan rapi.

An extension function is a member function of a class that is defined outside the class... This Extension function is not present in java... But Kotlin has given us the advantage of extension function which would eliminate the difficulties posed by Android framework [3].

Kalimat di atas ini menjelaskan bahwa dalam konsep OOP Kotlin, kita bisa menambahkan fungsi baru ke dalam sebuah class dari luar class tersebut tanpa harus mewarisinya, yang di mana fitur ini tidak ada di Java. Di Java, untuk menambah fungsionalitas kelas anak, pengembang biasanya harus membuat wrapper class tambahan, sementara Kotlin tidak menggunakan hal tersebut.

One of the most common pitfalls in many programming languages, including Java, is that accessing a member of a null reference will result in a null reference exception... Kotlin uses the feature called Null Safety to handle the NULL pointer situation. Unless it is explicitly called for, Kotlin does not throw a NullPointerException [3].

Kalimat di atas ini menjelaskan bahwa penanganan objek pada Java rentan menyebabkan crash pada aplikasi, atau NullPointerException jika objek tersebut bernilai null. OOP pada Kotlin didesain anti NullPointerException dengan Null Safety bawaan, jadi mengakses objek null tidak akan memutus alur eksekusi program atau menyebabkan error yang fatal.

Boiler-plate code can be avoided, e.g. we don't need to specify setters(), equals(), hashCode() or toString() methods. All these can be generated by Kotlin [3].

Kalimat di atas ini memberitahu bahwa pembuatan class atau objek di Java membutuhkan penulisan banyak kode teknis atau boilerplate, seperti setters(), equals(), dan lain-lain. OOP di Kotlin lebih bersih karena compiler Kotlin dapat men-generate metode-metode tersebut secara otomatis. Hal ini sangat bersifat efisien karena kelas bisa fokus pada data dan logikanya saja tanpa dipenuhi instruksi teknis.

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

The Single Responsibility Principle (SRP) advocates that each class only has responsibility for one part of the software's function; i.e. each class does just one thing, keeping the design as simple as possible, and future changes and updates easier and less disruptive [4].

Kalimat di atas ini menjelaskan bahwa setiap class di dalam kode hanya boleh memiliki satu tugas atau satu tanggung jawab utama. Dengan begitu, desain sistem akan tetap sederhana, jika di masa depan terjadi pembaruan fitur, perubahannya tidak akan mengganggu atau merusak fungsi lain di dalam sistem.

The Open-Closed Principle (OCP) emphasizes that classes should be open for extension but closed for modification. This means it is possible to extend a class's behavior without altering its source code [4].

Kalimat di atas ini menjelaskan bahwa sebuah class harus didesain agar kita bisa menambahkan fitur atau perilaku baru tanpa harus membongkar atau mengubah kode sumber aslinya yang sudah jadi. Prinsip ini sangat berguna untuk mengurangi risiko munculnya bug baru saat sistem dikembangkan.

...the Liskov Substitution Principle (LSP) asserts that derived classes should be replaceable for their parent classes without changing the function's correctness [4].

Kalimat di atas ini menjelaskan bahwa jika kita membuat sebuah class turunan, class tersebut harus bisa menggantikan posisi class induknya tanpa menyebabkan error atau mengubah perilaku program. Hal ini memastikan bahwa inheritance harus dibangun dengan benar dan konsisten.

The Interface Segregation Principle (ISP) encourages the creation of multiple, client-specific interfaces rather than attempting to extend the use of an existing interface. I.e., a client should never be forced to implement an interface that it doesn't use, or clients shouldn't be forced to depend on methods they do not use [4].

Kalimat di atas ini menjelaskan bahwa daripada membuat satu *interface* besar yang berisi banyak fungsi campuran, lebih baik memecahnya menjadi beberapa *interface* kecil yang spesifik untuk setiap kebutuhan. Hal ini bertujuan agar sebuah *class* tidak dipaksa menanggung semuanya untuk mengimplementasikan keseluruhan isi program.

The last one, called Dependency Inversion Principle (DIP), has the following premise: 'Depend on abstractions, not implementations'. The idea is that whenever it is necessary to couple to another class or module, the less stable one should depend on the more stable one [4].

Kalimat di atas ini menjelaskan bahwa komponen sistem harus bergantung pada lapisan abstraksi, seperti interface atau abstract class, bukan bergantung pada detail implementasi secara langsung. Hal ini bertujuan untuk mengurangi keterikatan antar modul.

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

A class which takes 50 lines of code in Java can be written in very few line in Kotlin. Boiler-plate code can be avoided, e.g. we don't need to specify setters(), equals(), hashCode() or toString() methods. All these can be generated by Kotlin [3].

Kelimat di atas menjelaskan bagaimana Kotlin menangani kelas yang fungsinya hanya untuk menampung data. Jadi perbedaan antasa class biasa dengan data class yaitu:

- Class Biasa: Pengembang harus mendefinisikan dan menulis sendiri kode-kode yang bersifat boilerplate secara manual. Jika sebuah kelas membutuhkan metode teknis untuk memanipulasi data di dalamnya, seperti setters(), equals(), hashCode(), atau toString(). Maka baris kode akan menjadi sangat panjang dan juga dipenuhi oleh detail teknis.
- Data Class: Class ini dirancang hanya untuk menyimpan data. Jika kita menggunakan penanda data class, Kotlin akan secara otomatis men-generate metode-metode boilerplate tersebut, seperti setters(), equals(), hashCode(), toString() di latar belakang. Maka, penulisan data class menjadi sangat ringkas, bersih, dan langsung berfokus pada struktur datanya saja.

E. Tautan Git

Berikut adalah tautan untuk repository yang dibuat untuk source code:

Link GitHub Modul 1

Daftar Pustaka

[1] R. Singh, R. Singh and A. Singh, "Comparative study of XML vs Jetpack Compose for UI Development in Android," *International Journal of Innovative Research in Technology,* pp. 6175-6184, 2025.

[2] P. Steinberger, "The Shift From Imperative to Declarative UI," 2021. [Online]. Available: https://increment.com/mobile/the-shift-to-declarative-ui/. [Accessed 4 April 2026].

[3] S. Bose, A. Kundu, M. Mukherjee and M. Banerjee, "A Comparative Study: Java Vs Kotlin Programming In Android Application Development," *International Journal of Advanced Research in Computer Science,* pp. 41-45, 2018.

[4] R. Cabral, H. Villamizar, M. Kalinowski, T. Escovedo, M. T. Baldassarre and H. Lopes, "Investigating the Impact of SOLID Design Principles on Machine Learning Code Understanding," *CAIN 2024,* 2024.